

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



Informe laboratorio 3

SIG

ICN-292

Alumna: Fernanda Sáez Cabrera

Fecha: 23 de septiembre 2026

Profesor: José Luis Sáez

Parámetros		
S	U	D
092	\$72.000	7 días

Repositorio GitHub: <https://github.com/fernandasaezc/ICN292-Lab3-Saez-Fernanda>

Resumen ejecutivo

El presente trabajo se centra en la automatización del proceso de devoluciones del canal e-commerce de AndesHogar SpA, mediante el desarrollo de un producto mínimo viable (MVP) en n8n. El flujo recibe solicitudes a través de un webhook, aplica las reglas de negocio para clasificarlas como aprobadas, rechazadas, en revisión o con datos inválidos, registra los resultados y entrega una respuesta. Además, se implementaron flujos complementarios para el envío de solicitudes y la generación de un resumen diario que se envía automáticamente por correo electrónico. Para la clasificación se utilizaron los parámetros personales $S = 92$, $U = \$72.000$ y $D = 7$ días.

De las 15 solicitudes evaluadas, 5 fueron aprobadas, 7 rechazadas y 3 derivadas a revisión, sin casos inválidos. La tasa de aprobación automática fue de 33,3%. El monto total del día alcanzó los \$1.251.820, distribuido en \$195.930 en aprobaciones, \$595.920 en rechazos y \$459.970 en revisiones. Los siete rechazos se debieron al incumplimiento del plazo establecido. Por su parte, los casos de control permitieron verificar el funcionamiento de las reglas de forma independiente de S: DEV-C01 fue aprobado, DEV-C02 pasó a revisión por monto y DEV-C03 fue rechazado por plazo.

También se comparó la implementación con Switch y Edit Fields con una alternativa basada en un nodo Code. Mientras la primera permite visualizar las distintas rutas del proceso, la segunda concentra la lógica y facilita su modificación. Se identificó una diferencia en el manejo de datos no numéricos: ante un monto como “veinte mil”, el flujo con Switch falla y detiene la ejecución, mientras que la versión con Code lo clasifica como dato inválido. Los resultados se respaldan mediante los tres workflows exportados, la tabla de clasificación, los registros individuales y las capturas de ejecuciones exitosas y fallidas. Todos los archivos se encuentran disponibles en el repositorio de GitHub: <https://github.com/fernandasaezc/ICN292-Lab3-Saez-Fernanda>.

En conclusión, el MVP demuestra que es posible automatizar el triage y mejorar la trazabilidad de las solicitudes. Se recomienda avanzar hacia su implementación en producción para la etapa de clasificación, pero no adoptarlo aún como sistema definitivo. Esto debido a que antes es necesario fortalecer la validación de entradas, incorporar un Error Workflow y reemplazar los registros locales por una base de datos con respaldo. Una vez implementadas estas

mejoras y verificado el funcionamiento de las integraciones, podría evaluarse su incorporación a la operación real de AndesHogar.

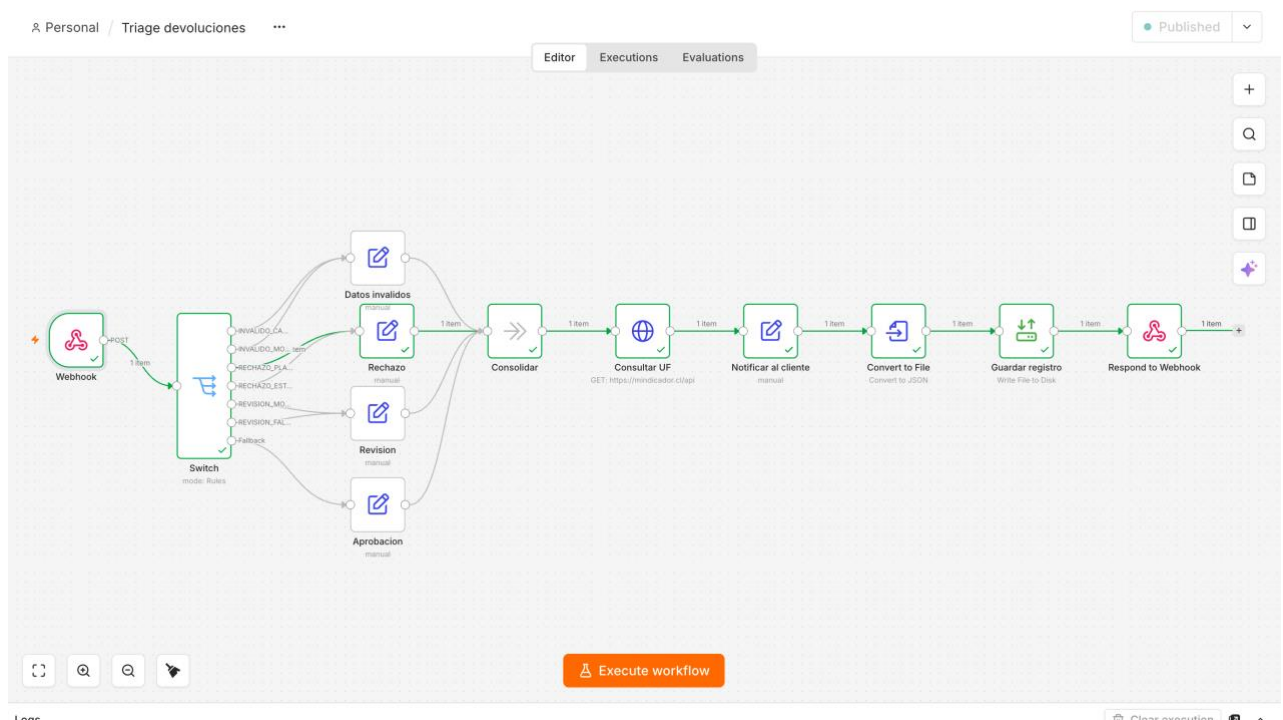
Parte A – Flujo de triage

El flujo principal de triage (ICN292-Lab3-Saez-Fernanda-triage.json) se construyó sin escribir código, utilizando doce nodos encadenados. La solicitud ingresa por un nodo Webhook configurado con método POST y el path triage-devoluciones, que responde mediante un nodo Respond to Webhook al final del proceso. Luego, un nodo Switch en modo Rules clasifica la solicitud aplicando las reglas de negocio en el orden de la política, con precedencia estricta: contiene seis reglas y una salida Fallback, y mantiene desactivada la opción "Send data to all matching outputs", de modo que gana la primera regla que se cumple. Las condiciones numéricas utilizan conversión de tipos para interpretar como números los montos y días, que llegan como texto.

Cada ruta posible se resuelve en uno de los cuatro nodos Edit Fields (Datos inválidos, Rechazo, Revisión y Aprobación), que escriben los campos de trazabilidad exigidos por la regla 7; id_solicitud, sku, monto, días, estado, ruta, motivo, U y D, derivando el motivo según la causal correspondiente. Las siete salidas del Switch se conectan a estos cuatro nodos: las dos causales de datos inválidos llegan a "Datos inválidos", las dos de rechazo a "Rechazo", las dos de revisión a "Revisión", y el Fallback a "Aprobación". El orden que ubica el rechazo antes que la revisión se conserva porque corresponde al orden de las reglas.

Posteriormente, un nodo No Operation, do nothing ("Consolidar") une las cuatro ramas en un único camino. El flujo consulta entonces el servicio público de indicadores económicos mediante un nodo HTTP Request con método GET a la URL <https://mindicador.cl/api>, sin autenticación. De la respuesta se obtiene el valor el campo uf.valor, con el cual se calcula el monto expresado en UF ($\text{monto} \div \text{uf.valor}$). En la ejecución realizada, la UF tenía un valor de \$40.983,58. Finalmente, un nodo Edit Fields (Set), llamado "Notificar al cliente", arma la notificación incorporando el valor de la UF y el monto convertido. Este nodo prepara la notificación, pero no la envía, ya que los correos de las solicitudes son ficticios y el envío rebotaría. En una implementación real se conectaría a un nodo Send Email dirigido al campo email_cliente de cada solicitud. Luego, dos nodos (Convert to File y Guardar

registro) convierten el resultado en JSON y lo escriben en disco generando un archivo por solicitud, de modo que cada decisión queda registrada y puede revisarse posteriormente. Por su parte, el nodo Respond to Webhook devuelve al emisor una respuesta con el identificador, la ruta, el motivo, el monto y el monto en UF. Lo mencionado anteriormente se puede observar en la siguiente captura del flujo de triage completo ejecutado con éxito:



El nodo Switch evalúa las reglas en el siguiente orden, deteniéndose en la primera que se cumple:

Condición	Salida
id_solicitud está vacío	INVALIDO_CAMPO
monto ≤ 0	INVALIDO_MONTO
dias_desde_compra > 7 (D)	RECHAZO_PLAZO
estado_producto = danado_por_uso	RECHAZO_ESTADO
monto > 72.000 (U)	REVISION_MONTO
estado_producto = con fallas	REVISION_FALLA
Fallback	APROBACION

Respecto a las URLs del webhook, n8n genera dos con comportamientos distintos. Dado que el webhook usa método POST, no puede activarse pegando la URL en el navegador, ya que este solo realiza peticiones GET. Es por esto que las solicitudes se envían mediante peticiones POST con los datos de la solicitud en formato JSON. La URL de prueba (<http://localhost:5678/webhook-test/triage-devoluciones>) se activa al presionar 'Listen for test event', muestra los

datos de la ejecución en el editor y atiende una sola llamada por activación, por lo que resulta conveniente mientras se arma el flujo y al momento de registrar la información de entrada y salida de cada nodo. La URL de producción (<http://localhost:5678/webhook/triage-devoluciones>) se activa al publicar el workflow, admite llamadas sucesivas y se ejecuta en segundo plano. En este trabajo se utilizó la URL de prueba para las capturas de ejecuciones individuales, donde interesa visualizar la entrada y salida de cada nodo, y la URL de producción para procesar las 15 solicitudes de corrido mediante el flujo emisor, dado que la de prueba solo atiende una llamada por activación.

Al procesar las 15 solicitudes, se obtuvo la siguiente clasificación:

id_solicitud	monto	días	estado_producto	ruta	motivo
DEV-1115	\$19.990	3	sin abrir	APROBACION	APROBACION
DEV-1116	\$69.980	5	abierto sin uso	APROBACION	APROBACION
DEV-1117	\$24.990	12	con fallas	RECHAZO	RECHAZO_PLAZO
DEV-1118	\$79.990	4	sin abrir	REVISION	REVISION_MONTO
DEV-1119	\$45.990	25	danado por uso	RECHAZO	RECHAZO_PLAZO
DEV-1120	\$89.990	2	con fallas	REVISION	REVISION_MONTO
DEV-1121	\$289.990	9	sin abrir	RECHAZO	RECHAZO_PLAZO
DEV-1122	\$45.990	2	sin abrir	APROBACION	APROBACION
DEV-1123	\$39.980	15	danado por uso	RECHAZO	RECHAZO_PLAZO
DEV-1124	\$49.980	1	abierto sin uso	APROBACION	APROBACION
DEV-1125	\$34.990	22	sin abrir	RECHAZO	RECHAZO_PLAZO
DEV-1126	\$79.990	8	con fallas	RECHAZO	RECHAZO_PLAZO
DEV-C01	\$9.990	1	sin abrir	APROBACION	APROBACION
DEV-C02	\$289.990	2	sin abrir	REVISION	REVISION_MONTO
DEV-C03	\$79.990	29	abierto sin uso	RECHAZO	RECHAZO_PLAZO

The screenshot shows a webhooks testing tool interface. The left sidebar lists input parameters: headers (host, user-agent, accept, content-type, content-length), params, query, and body. The main area is titled 'Manual Mapping' and shows a table of fields to be set. The right sidebar displays the output JSON for a single item.

Field	Type	Value
id_solicitud	T String	DEV-1117
sku	T String	HV-050
monto	# Number	24990
dias_desde_compra	# Number	12
estado_producto	T String	con_fallas
email_cliente	T String	c3@example.cl

The output JSON on the right shows the following structure:

```

{
  "id_solicitud": "DEV-1117",
  "sku": "HV-050",
  "monto": 24990,
  "dias_desde_compra": 12,
  "estado_producto": "con_fallas",
  "email_cliente": "c3@example.cl",
  "ruta": "RECHAZO",
  "motivo": "RECHAZO_PLAZO",
  "U": 72000,
  "D": 7
}

```

En resumen, se registraron 5 aprobaciones por un total de \$195.930, 7 rechazos por \$595.920 y 3 revisiones por \$459.970, alcanzando un monto total del día de \$1.251.820 y una tasa de aprobación automática de 33,3%. Cabe destacar que, con D = 7 días, los siete rechazos se produjeron por incumplimiento del plazo. Debido a la precedencia de las reglas, las solicitudes con estado danado_por_uso o con_fallas que además superaban el plazo o el monto límite fueron clasificadas antes por esas causales, por lo cual en este caso no se activaron los motivos RECHAZO_ESTADO ni REVISION_FALLA.

Parte B – Flujo programado de resumen

El tercer workflow (ICN292-Lab3-Saez-Fernanda-resumen.json) genera automáticamente un resumen diario de las devoluciones procesadas. Comienza con un nodo Schedule Trigger, que ejecuta el flujo de forma programada. Luego sigue un nodo Read/Write Files from Disk que lee todos los registros almacenados por el flujo de triage, y un nodo Extract from File recupera el contenido de cada archivo. Como los datos se obtienen agrupados dentro de un campo llamado “data”, se incorporó un nodo Edit Fields que los reorganiza para dejar los campos de cada solicitud (identificador, ruta, monto, entre otros) directamente disponibles, de modo que puedan ser procesados.

El nodo Summarize consolida la información agrupando las solicitudes por ruta, contando la cantidad de cada tipo y sumando sus montos. Finalmente, un nodo Edit Fields arma un único mensaje de texto con toda la información solicitada, la cantidad de solicitudes por ruta, monto total por ruta, monto total del día y tasa de aprobación automática. Este nodo se configuró con la opción Execute Once, para generar un único mensaje y no uno por cada ruta. Luego, un nodo Send Email envía el resumen por correo electrónico mediante el servidor SMTP de Gmail, utilizando una credencial guardada en el gestor de n8n, de modo que la contraseña no queda incluida en el archivo .json exportado. Cabe señalar que el nodo de lectura toma todos los archivos de la carpeta de registros sin filtrar por fecha, por lo que en operación sería necesario filtrar por día o archivar los registros después de cada resumen.

El correo recibido en la ejecución fue el siguiente:

- Resumen diario AndesHogar
APROBACIÓN: 5 solicitudes, monto \$195.930
RECHAZO: 7 solicitudes, monto \$595.920
REVISIÓN: 3 solicitudes, monto \$459.970
Monto total del día: \$1.251.820
Tasa de aprobación automática: 33,3%

Las siguientes capturas muestran el nodo Send Email con su entrada y salida, y el correo recibido:

En cuanto a la frecuencia, se configuró el Schedule Trigger para ejecutarse una vez al día a las 20:00 horas. Se eligió este horario porque el canal e-commerce recibe solicitudes a lo largo de la jornada, de modo que ejecutar el resumen al final del día permite considerar todas las solicitudes recibidas y dejar el reporte disponible para el otro día. Respecto a qué ocurre un día sin solicitudes, el flujo igualmente se ejecuta a la hora programada, pero al no encontrar registros el nodo de lectura no entrega datos y los nodos siguientes no se ejecutan, por lo que no se envía ningún correo. Esto genera un problema, ya que la ausencia del

resumen no permite distinguir entre un día sin devoluciones y una falla del flujo. Como mejora, se podría incorporar un nodo condicional que, al no encontrar registros, envíe igualmente un correo indicando "sin solicitudes en el día".

Parte C – Errores y casos bordes

Para evaluar el comportamiento del flujo ante situaciones anormales, se probaron distintas entradas inválidas y casos límite, registrando en cada caso la carga enviada, el comportamiento esperado y el observado.

Las 3 entradas inválidas fueron las siguientes:

1- Identificador vacío.

Se envió una solicitud con el campo `id_solicitud` vacío. Se esperaba que fuera clasificada como `DATOS_INVÁLIDOS` con motivo `INVALIDO_CAMPO`, ya que la primera regla del Switch detecta este caso. El comportamiento observado coincidió con lo esperado.

Carga enviada: `{"id_solicitud":"","sku":"PL-120","monto":"45990","dias_desde_compra":"2","estado_producto":"sin_abrir"}`

Resultado:

The screenshot shows a webhooks testing tool interface. On the left, the 'INPUT' tab is active, displaying a 'Switch' node with a 'Manual Mapping' mode. The 'body' section of the input shows a JSON object: `{ "id_solicitud": "", "sku": "PL-120", "monto": "45990", "dias_desde_compra": "2", "estado_producto": "sin_abrir" }`. The 'Parameters' tab is also visible, showing fields like `id_solicitud`, `sku`, `monto`, `dias_desde_compra`, `estado_producto`, and `email_cliente` with their respective values. On the right, the 'OUTPUT' tab shows the result: `{ "id_solicitud": "", "sku": "PL-120", "monto": "45990", "dias_desde_compra": "2", "estado_producto": "sin_abrir", "email_cliente": null, "ruta": "DATOS_INVALIDOS", "motivo": "INVALIDO_CAMPO", "U": "72000", "D": "7" }`.

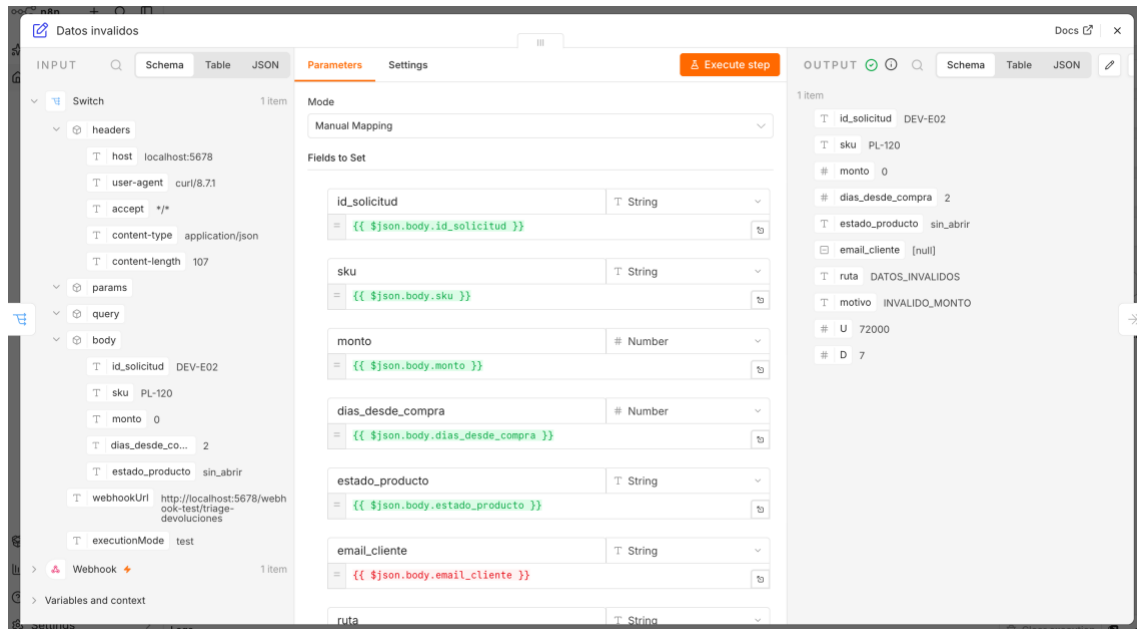
2- Monto igual a cero.

Se envió una solicitud con `monto` igual a `"0"`. Se esperaba la clasificación `DATOS_INVÁLIDOS` con motivo `INVALIDO_MONTO`, por no

cumplir la condición de que el monto sea mayor que cero. El comportamiento observado coincidió con lo esperado.

Carga enviada: {"id_solicitud":"DEV-E02", "sku":"PL-120", "monto":"0", "dias_desde_compra":"2", "estado_producto":"sin_abrir"}

Resultado:



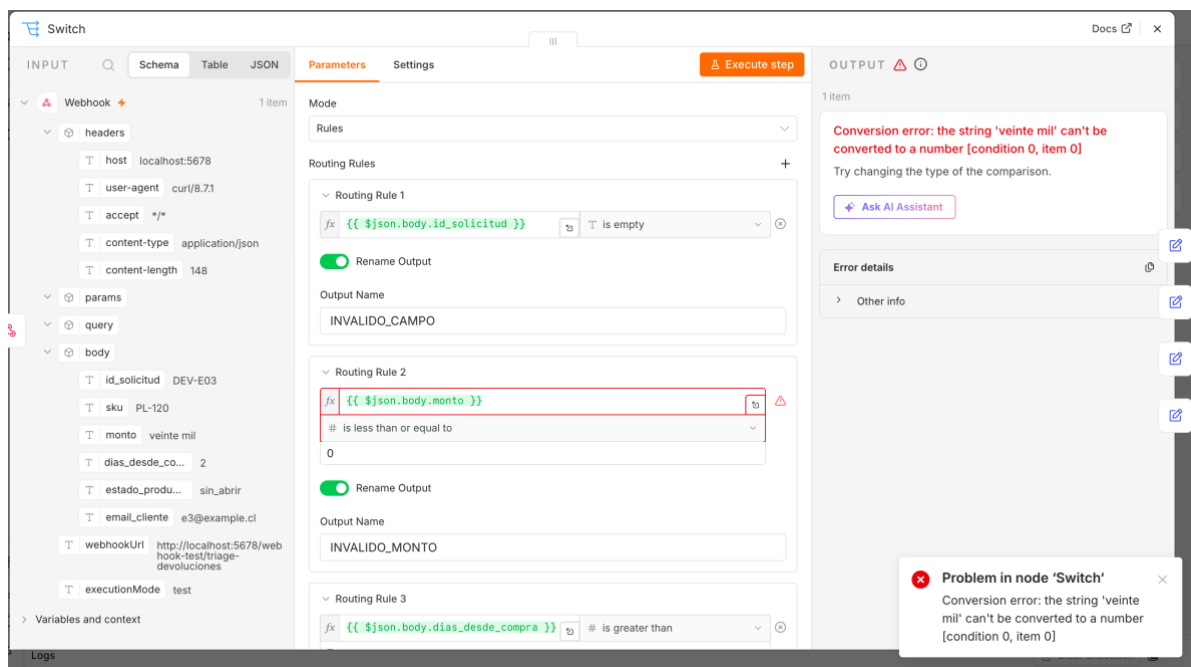
3- Monto no numérico.

Se envió una solicitud con el campo monto escrito en palabras ("veinte mil"). A diferencia de los casos anteriores, esta entrada no toma una ruta de clasificación, sino que provoca la falla del nodo Switch. Al intentar convertir "veinte mil" a número para evaluar las condiciones de monto, la conversión no es posible y el nodo arroja un error, y el flujo se detiene. Esto cumple con el requisito de que al menos una de las entradas inválidas haga fallar un nodo, y no solo tome la ruta de datos inválidos.

Carga enviada:

{"id_solicitud":"DEV-E03", "sku":"PL-120", "monto":"veinte mil", "dias_desde_compra":"2", "estado_producto":"sin_abrir"}

Resultado:



En cuanto al manejo de errores, cada nodo en n8n permite elegir entre tres valores para "On Error": "Stop Workflow", que detiene la ejecución, "Continue", que permite seguir entregando el error como salida normal, y "Continue (using error output)", que continúa dirigiendo el elemento con error a una salida separada. En este trabajo se mantuvo "Stop Workflow" en todos los nodos, por razones distintas según el nodo. En el Switch, porque un dato que no puede convertirse no debe clasificarse con una ruta equivocada. En Consultar UF, porque si la consulta falla la notificación y el registro quedarían sin el monto en UF. Como mejora futura, este nodo podría usar "Continue (using error output)" para que una caída de mindicador.cl no impida clasificar la solicitud. En Guardar registro, porque una decisión que no queda registrada no es auditable según la regla 7. Y en Send Email, para que una falla en el envío del resumen quede visible en lugar de pasar desapercibida.

Relacionado con lo anterior, un Error Workflow es un flujo independiente que se ejecuta automáticamente cuando otro falla. Comienza con un nodo Error Trigger y recibe el nombre del flujo, el nodo que falló y el mensaje de error, permitiendo enviar una alerta o registrar el incidente. Es especialmente importante en un flujo programado como el resumen diario, al ejecutarse sin supervisión, si un día falla nadie lo notaría, ya que simplemente no llegaría el resumen, sin ningún otro aviso. El flujo podría dejar de funcionar sin que nadie

se dé cuenta. Un Error Workflow evita esto al convertir cada fallo en una alerta activa.

Casos límites y pruebas de robustez.

Dado que las 15 solicitudes no incluyen casos en el límite exacto de los parámetros, se construyeron dos adicionales para verificar las comparaciones inclusivas (regla 6). Una solicitud con monto igual a \$72.000 (U) fue clasificada como APROBACIÓN, ya que la regla de revisión exige que el monto sea estrictamente mayor que U.

El resultado se observa a continuación:

The screenshot displays a workflow configuration tool for a process named 'Aprobacion'. It is divided into three main sections: INPUT, Parameters, and OUTPUT.

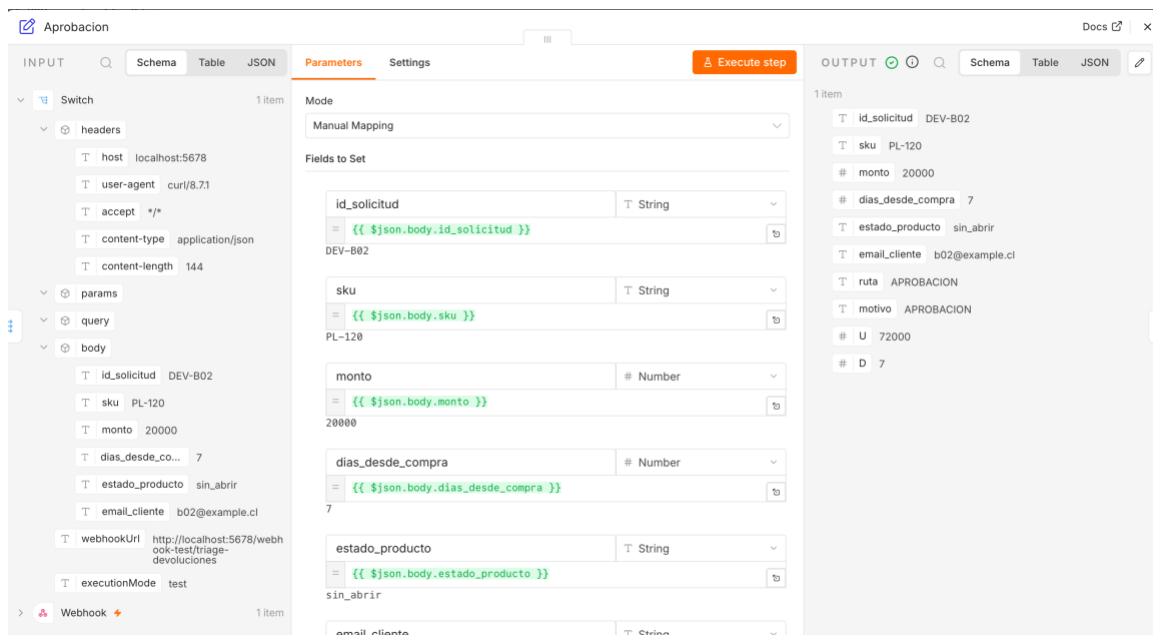
INPUT Section: Shows a 'Switch' configuration with a 'headers' tab. The headers include: host (localhost:5678), user-agent (curl/8.7.1), accept (*/*), content-type (application/json), and content-length (144). Below this, a 'params' tab shows the following parameters: id_solicitud (DEV-B01), sku (PL-120), monto (72000), dias_desde_compra (2), estado_producto (sin_abrir), email_cliente (b01@example.cl), webhookUrl (http://localhost:5678/webhook-test/triage-devoluciones), and executionMode (test).

Parameters Section: Shows a 'Manual Mapping' mode. The 'Fields to Set' are configured as follows: id_solicitud (T String) is mapped to {{ \$json.body.id_solicitud }}; sku (T String) is mapped to {{ \$json.body.sku }}; monto (# Number) is mapped to {{ \$json.body.monto }}; dias_desde_compra (# Number) is mapped to {{ \$json.body.dias_desde_compra }}; estado_producto (T String) is mapped to {{ \$json.body.estado_producto }}; and email_cliente (T String) is mapped to {{ \$json.body.email_cliente }}.

OUTPUT Section: Shows the result of the workflow execution. The output is a JSON object with the following fields: id_solicitud (DEV-B01), sku (PL-120), monto (72000), dias_desde_compra (2), estado_producto (sin_abrir), email_cliente (b01@example.cl), ruta (APROBACION), motivo (APROBACION), D (72000), and D (7).

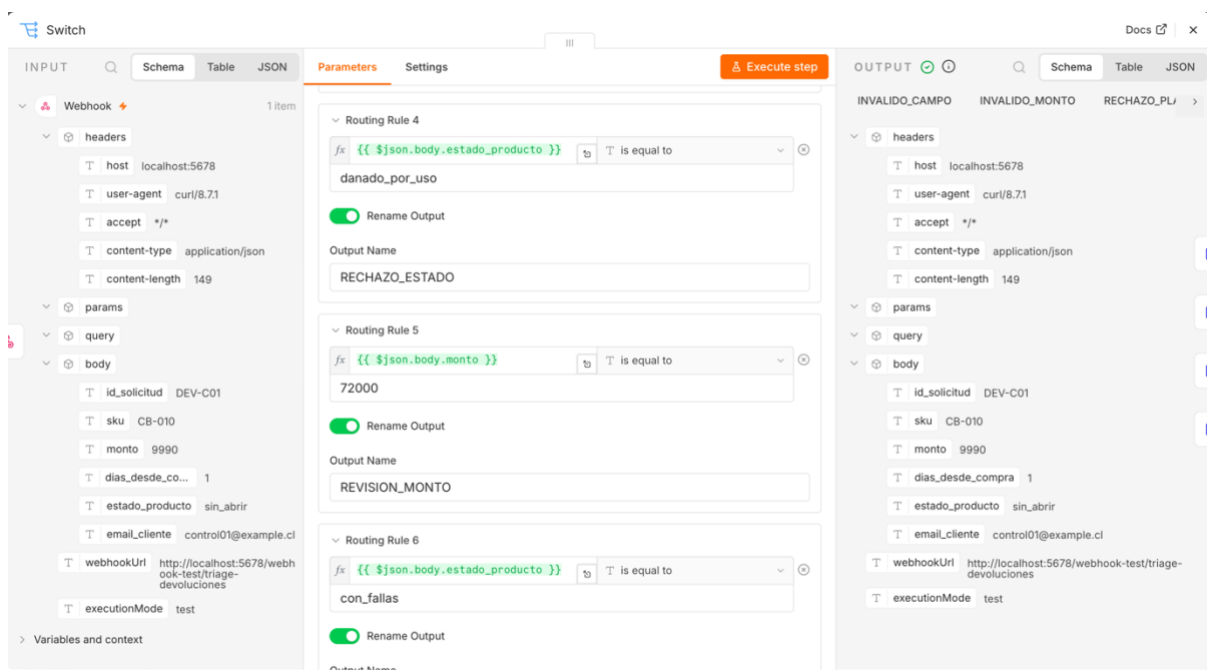
Del mismo modo, una solicitud con dias_desde_compra igual a 7 (D) también fue aprobada, dado que el rechazo por plazo requiere superar D. Ambos resultados confirman que un monto igual a U se aprueba y un plazo igual a D se considera dentro de plazo.

El resultado se observa en la siguiente captura:



También se realizó la prueba de error de tipo, se cambió la condición del monto de Number a String y se envió DEV-C01 (monto \$9.990), que fue clasificada como APROBACIÓN. Como texto, la condición solo permite "is equal to", por lo que "9990" no coincide con "72000" y la solicitud cae en el Fallback; el control de montos altos deja de funcionar sin mostrar ningún error. Luego se revirtió la condición a Number.

El resultado se puede observar en la siguiente captura:



Casos que el flujo no puede resolver.

Finalmente, se identificaron dos situaciones que el flujo actual no logra manejar. La primera es un monto en palabras como "veinte mil": al no poder convertirse a número, el Switch falla y detiene la ejecución (situación mostrada anteriormente en las entradas inválidas), sin distinguir un dato mal ingresado de un error del sistema; para detectarla se necesitaría una validación previa que verifique que el monto contiene solo dígitos antes de evaluarlo.

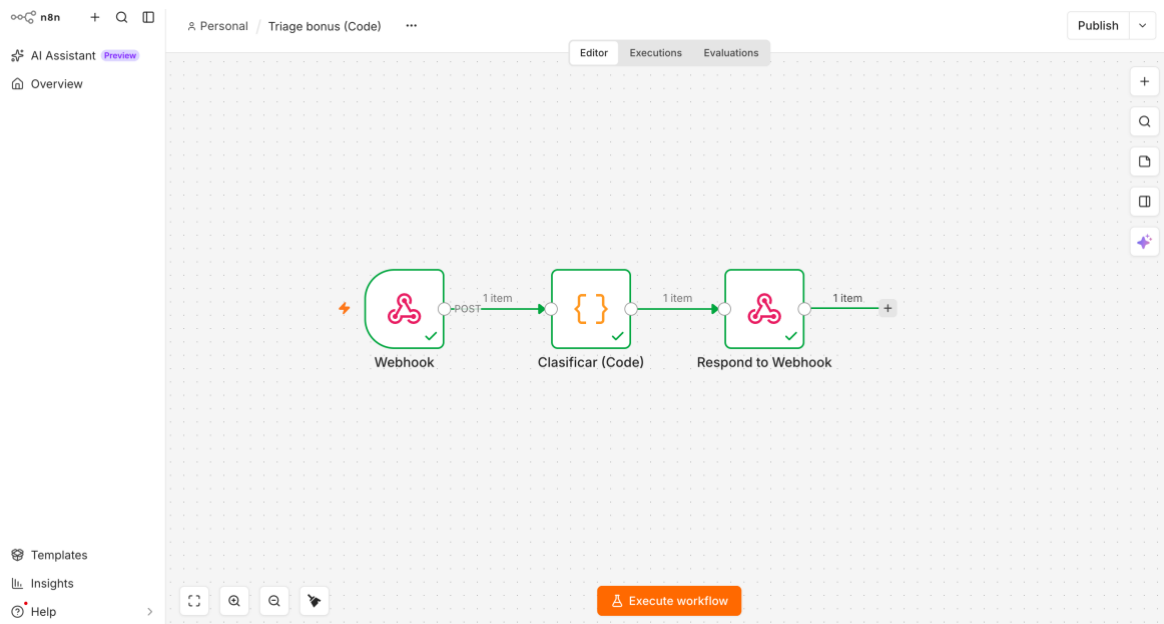
En este caso, se ingresó una solicitud con el SKU ZZ-999. Como se observa en la siguiente imagen, el flujo procesa la solicitud y la clasifica según el monto, los días transcurridos desde la compra y el estado del producto, ya que no verifica si el SKU existe en el catálogo. Para evitar esto, sería necesario incorporar una validación que compruebe que el código exista antes de continuar con el proceso.

The screenshot displays a workflow configuration in a tool like n8n. The 'INPUT' section on the left lists headers (host, user-agent, accept, content-type, content-length) and body parameters (id_solicitud, sku, monto, dias_desde_compra, estado_producto, email_cliente, webhookUrl, executionMode). The 'Parameters' section in the center shows 'Fields to Set' for id_solicitud, sku, monto, dias_desde_compra, estado_producto, and email_cliente, each mapped to a JSON path like {{ \$json.body.id_solicitud }}. The 'OUTPUT' section on the right shows the resulting JSON object with values like id_solicitud: DEV-SKU01, sku: ZZ-999, monto: 19990, dias_desde_compra: 3, estado_producto: sin_abrir, email_cliente: cliente@example.cl, ruta: APROBACION, motivo: APROBACION, U: 72000, D: 7.

Bonus - Implementación con un nodo Code

Como alternativa a la implementación principal, se construyó una versión del triage que reemplaza el nodo Switch y los cuatro nodos Edit Fields por un único nodo Code. Este nodo concentra toda la lógica de clasificación en una función escrita en JavaScript, que evalúa las reglas en el mismo orden de precedencia y devuelve la ruta, el motivo y los campos de trazabilidad. Al enviar las solicitudes de prueba, esta versión entregó exactamente los mismos resultados

que la implementación con Switch, lo que confirma que ambas son equivalentes. El flujo con un único nodo Code se muestra a continuación:



Respecto a cuál es más fácil de auditar, la implementación con Switch tiene ventaja ya que sus reglas, su orden y sus salidas se visualizan directamente en el editor, de manera que una persona sin conocimientos de programación puede revisar la política de negocio sin leer código. En la versión con Code, en cambio, es necesario leer JavaScript para verificar que la precedencia y las condiciones sean correctas.

Por su parte, respecto a cuál es más fácil de mantener, la versión con Code resulta más cómoda ante cambios en la lógica, cambiar un parámetro como U o D es cambiar una línea, y agregar una regla es sumar una condición más. En el Switch, en cambio, un cambio implica cambiar la condición, la salida y su conexión al Edit Fields correspondiente, con mayor riesgo de alterar el orden en que se evalúan las reglas

Sobre cuál elegiría si la política cambiara cada mes, la respuesta depende de quién realice los cambios. Si los realiza una persona con conocimientos técnicos, la versión con Code es preferible, ya que permite ajustar las reglas rápidamente y revisar los cambios de forma centralizada, idealmente, los parámetros (U, D y el orden) se externalizarían para que la modificación mensual sea solo de datos y no de lógica. Por otro lado, si los cambios los realiza alguien del área de negocio sin conocimientos técnicos, el Switch sigue siendo preferible a pesar de su mayor costo de edición, por su claridad visual.

Finalmente, durante las pruebas se observó una diferencia importante en el manejo de datos no numéricos. Al enviar una solicitud con monto "veinte mil", la versión con Switch falla y detiene la ejecución, mientras que la versión con Code la clasifica como dato inválido (INVALIDO_MONTO) ya que, al no poder convertir el texto a número, el valor resultante no cumple la condición de ser mayor que cero. Esto da cuenta que la implementación con Code es más estable ante datos mal formados, aunque esa mayor tolerancia también implica que un error de entrada puede pasar desapercibido en vez de detenerse de forma visible.